

1. True or False.

This question is related to the example using a payroll thread and an ATM withdrawal thread. If the two threads fully execute sequentially (i.e., one thread fully executes followed by the execution of the other thread), the integrity of the shared variable B (balance) may get violated.

False

2. The integrity of the variables may be violated means:

variables may be set to incorrect values

variables may take unintended values

3. A solution to mutual exclusion must have the following components: _____, entry code, and exit code.

initialization

4. To be a critical region,

at least two threads have some instructions accessing shared data

at least one thread have some instructions writing some shared data while another may be writing or reading the same shared data

5. The solution below does NOT fulfill the progress requirement because:

Initialization: Turn = 1	
<pre>while (1){ while (turn != 1); Critical Section turn = 0; Remainder Section }</pre>	<pre>while (1){ while (turn != 0); Critical Section turn = 1; Remainder Section }</pre>

The threads MUST keep entering alternatively (in turns) to ensure that they can enter their critical regions.

6. Select the best option to complete code for the first attempt to provide mutual exclusion. Let us limit variables to take only values 0 or 1.

Initialization: Turn = 1	
<pre>while (1){ while (turn != 1); Critical Section [Select] Remainder Section }</pre>	<pre>while (1){ while (turn == 1); Critical Section [Select] Remainder Section }</pre>

turn = 0

==

turn = 1

7. Select the best option to complete code for the second attempt to provide mutual exclusion.

Initialization: Interested0 = Interested1 = false	
<pre>while (1){ Interested1 = true; while ([Select]); Critical Section Interested1 = false Remainder Section }</pre>	<pre>while (1){ Interested0 = true while ([Select]); Critical Section Interested0 = false Remainder Section }</pre>

Interested0
Interested1 = false
Interested0 = true
Interested1

8. Select which thread(s) will get to the critical region first.

Initialization: Interested0 = true; Interested1 = true	
<pre>while (1){ Interested0 = true; while (Interested1); Critical Section Interested0 = false Remainder Section }</pre>	<pre>while (1){ Interested1 = true; while (Interested0); Critical Section Interested1 = false Remainder Section }</pre>

None will reach its critical region

9. Select the best option to complete code for the second attempt to provide mutual exclusion. If possible, we want to guarantee that the thread on the left will enter first to the critical region. if impossible, choose for all choices "No guarantee".

Initialization: Interested0 = No guarantee ; Interested1 = false	
<pre>while (1){ Interested0 = true; while (Interested1); Critical Section Interested0 = false Remainder Section }</pre>	<pre>while (1){ Interested1 = true; while (Interested0); Critical Section Interested1 = false Remainder Section }</pre>

?
?

10. The solution below meets the following requirements.

Initialization: Interested0 = false; Interested1 = false	
<pre> while (1){ Interested0 = true; turn = 1; while (Interested1 && turn == 1); Critical Section Interested0 = false Remainder Section } </pre>	<pre> while (1){ Interested1 = true; turn = 0; while (Interested0 && turn == 0); Critical Section Interested1 = false Remainder Section } </pre>

Mutual Exclusion
?

11. Select the best option to complete code for the Peterson Solution to provide mutual exclusion.

Initialization: a = [Select] ; b = [Select] ;	
<pre> while (1){ a = true; ; turn = 1; while ([Select]) && [Select]); Critical Section a = false; Remainder Section } </pre>	<pre> while (1){ [Select] ; [Select] ; while ([Select]) && turn == 0); Critical Section [Select] Remainder Section } </pre>

false
 false
 true
 b
 (turn == 1)
 b = true
 turn = 0
 a
 turn == 0
 b = false